

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи № 10
з курсу “Операційні системи реального часу”
на тему: “Розробка багатозадачної інтерактивної гри для міні-консолі”

Виконав:
Студент. гр. ФеП-41 Фіняк Олег
Перевірив:
Павлик М.Р.

Львів – 2025

Мета роботи. Навчитися розробляти прості інтерактивні ігри типу “пінг-понг” та “змійка” для міні-консолі з використанням механізмів RTOS.

```
/**
 * @file lab-10.c
 * @brief FreeRTOS-based multi-game system for Raspberry Pi Pico with LCD display
 *
 * This program implements a real-time multi-game system using FreeRTOS on a Raspberry Pi Pico.
 * It consists of multiple tasks demonstrating:
 * - FreeRTOS Tasks: Separate tasks for input handling, game logic, and display rendering
 * - FreeRTOS Queues: Message passing between tasks for input events
 * - FreeRTOS Semaphores: Synchronization for frame buffer access and display updates
 * - LCD Display: Real-time rendering of game graphics on 1.3 inch LCD screen
 * - Multiple Games: Snake game, Math Quiz game, and menu system
 *
 * The system uses FreeRTOS queues to decouple input reading from game logic, and semaphores
 * to protect shared frame buffer access. Each game runs in a dedicated game logic task that
 * processes input events and updates the display. The display task handles LCD refresh timing.
 *
 * @author Mykyta Siryk
 * @date 2025-12-03
 * @version 1.0
 */

// Standard C library for input/output operations and formatted string functions
#include <stdio.h>
// Standard C library for memory allocation functions (malloc, free)
#include <stdlib.h>
// Standard C library for string manipulation functions (memcmp, etc.)
#include <string.h>
// Raspberry Pi Pico SDK for hardware access (GPIO, timing, etc.)
#include "pico/stdlib.h"
// FreeRTOS core header - provides task management, scheduling, etc.
#include "FreeRTOS.h"
// FreeRTOS task management functions (xTaskCreate, vTaskDelay, etc.)
#include "task.h"
// FreeRTOS queue management functions (xQueueCreate, xQueueSend, xQueueReceive, etc.)
#include "queue.h"
// FreeRTOS semaphore and mutex management functions (xSemaphoreCreate, etc.)
#include "semphr.h"
// Waveshare LCD display driver - device configuration and initialization
#include "DEV_Config.h"
// Waveshare LCD display driver - graphics and drawing functions
#include "GUI_Paint.h"
// Waveshare LCD display driver - 1.3 inch LCD display controller
#include "LCD_1in3.h"

/* ===== GPIO Pin Definitions ===== */

/** @brief GPIO pin number for button A input */
#define BTN_A 15
/** @brief GPIO pin number for button B input */
#define BTN_B 17
/** @brief GPIO pin number for directional button UP input */
#define BTN_UP 2
/** @brief GPIO pin number for directional button DOWN input */
#define BTN_DOWN 18
/** @brief GPIO pin number for directional button LEFT input */
#define BTN_LEFT 16
/** @brief GPIO pin number for directional button RIGHT input */
#define BTN_RIGHT 20
/** @brief GPIO pin number for control button input */
#define BTN_CTRL 3
```

```

/* ===== Timing Constants ===== */

/** @brief Button debounce delay time in milliseconds */
#define DEBOUNCE_TIME 50

/* ===== Color Definitions (RGB565 format) ===== */

/** @brief Background color (white) */
#define COLOR_BG WHITE
/** @brief Foreground color (black) */
#define COLOR_FG BLACK
/** @brief Obstacle/enemy color (red) */
#define COLOR_OBSTACLE 0xF800
/** @brief Snake color (blue) */
#define COLOR_SNAKE 0x001F
/** @brief Food color (red) */
#define COLOR_FOOD 0xF800

/* ===== Data Structures ===== */

/**
 * @brief Structure representing the current state of all input buttons
 *
 * This structure stores the pressed/released state of all game control buttons.
 * Each boolean field indicates whether the corresponding button is currently pressed.
 */
typedef struct {
    bool btnA, btnB, btnUp, btnDown, btnLeft, btnRight, btnCtrl;
} InputState;

/**
 * @brief Enumeration of available game types
 *
 * Defines the different game modes available in the system:
 * - GAME_MENU: Main menu screen
 * - GAME_MATH_QUIZ: Math quiz game
 * - GAME_SNAKE: Snake game
 */
typedef enum { GAME_MENU, GAME_MATH_QUIZ, GAME_SNAKE } GameType;

/* ===== Global Variables ===== */

/** @brief Pointer to the LCD frame buffer for drawing operations */
UWORD *frameBuffer;
/** @brief FreeRTOS queue handle for input state messages from input task to game logic task */
QueueHandle_t inputQueue;
/** @brief FreeRTOS binary semaphore handle for signaling when a new frame is ready to display */
SemaphoreHandle_t frameSemaphore;
/** @brief FreeRTOS mutex handle for protecting frame buffer access from concurrent tasks */
SemaphoreHandle_t bufferMutex;
/** @brief Current active game type/mode */
GameType currentGame = GAME_MENU;
/** @brief High score for Snake game */
int snakeHighScore = 0;

/* ===== SNAKE GAME ===== */

/** @brief Maximum length of the snake (maximum number of segments) */
#define SNAKE_MAX 100
/** @brief Size of each grid cell in pixels */
#define GRID_SIZE 12
/** @brief Width of the game grid in cells */
#define GRID_W 20
/** @brief Height of the game grid in cells */
#define GRID_H 20

```

```

/**
 * @brief Structure representing the complete state of the Snake game
 *
 * This structure stores all game state including snake position, direction,
 * food location, score, and game over status.
 */
typedef struct {
    int x[SNAKE_MAX], y[SNAKE_MAX]; /**< Array of x,y coordinates for each snake segment */
    int len; /**< Current length of the snake (number of segments) */
    int dir; /**< Current direction (0=right, 1=down, 2=left, 3=up) */
    int foodX, foodY; /**< Grid coordinates of the food */
    int score; /**< Current game score */
    bool gameOver; /**< Flag indicating if the game is over */
} SnakeGame;

/** @brief Global instance of the Snake game state */
SnakeGame snake;

/**
 * @brief Spawn food at a random location that doesn't overlap with the snake
 *
 * This function randomly places food on the game grid, ensuring it doesn't
 * spawn on any position currently occupied by the snake's body segments.
 * The function uses a validation loop to find a valid empty position.
 *
 * @note The food position is stored in snake.foodX and snake.foodY
 * @note Uses rand() for random number generation
 */
void spawnFood() {
    bool valid;
    // Keep trying random positions until we find one that doesn't overlap with snake
    do {
        valid = true;
        // Generate random grid coordinates
        snake.foodX = rand() % GRID_W;
        snake.foodY = rand() % GRID_H;

        // Check if the random position overlaps with any snake segment
        for (int i = 0; i < snake.len; i++) {
            if (snake.x[i] == snake.foodX && snake.y[i] == snake.foodY) {
                valid = false;
                break;
            }
        }
    } while (!valid);
}

/**
 * @brief Initialize the Snake game to starting state
 *
 * This function resets the snake game to its initial state:
 * - Sets snake length to 3 segments
 * - Positions snake horizontally at (5,5), (4,5), (3,5)
 * - Sets initial direction to right (0)
 * - Resets score to 0
 * - Clears game over flag
 * - Spawns initial food
 */
void initSnake() {
    // Initialize snake with 3 segments
    snake.len = 3;
    // Set initial direction to right (0 = right, 1 = down, 2 = left, 3 = up)
    snake.dir = 0;
    // Position snake horizontally: head at (5,5), body segments at (4,5) and (3,5)

```

```

snake.x[0] = 5; snake.y[0] = 5;
snake.x[1] = 4; snake.y[1] = 5;
snake.x[2] = 3; snake.y[2] = 5;
// Reset game state
snake.score = 0;
snake.gameOver = false;
// Spawn initial food at a random valid location
spawnFood();
}

/**
 * @brief Update Snake game state based on input and game logic
 *
 * This function processes one game tick of the Snake game:
 * - Handles game over state (restart on button press)
 * - Processes directional input (prevents 180-degree turns)
 * - Moves snake by shifting body segments and updating head position
 * - Checks for wall collisions
 * - Checks for self-collision
 * - Handles food consumption (grow snake, increase score, spawn new food)
 *
 * @param input Pointer to InputState structure containing current button states
 *
 * @note Direction values: 0=right, 1=down, 2=left, 3=up
 * @note Prevents reversing direction (e.g., can't go up if currently going down)
 */
void updateSnake(InputState *input) {
    // If game is over, check for restart input
    if (snake.gameOver) {
        if (input->btnA || input->btnUp) initSnake();
        return;
    }

    // Process directional input (prevent 180-degree turns)
    // Only change direction if not opposite to current direction
    if (input->btnUp && snake.dir != 1) snake.dir = 3; // Up (if not going down)
    if (input->btnDown && snake.dir != 3) snake.dir = 1; // Down (if not going up)
    if (input->btnLeft && snake.dir != 0) snake.dir = 2; // Left (if not going right)
    if (input->btnRight && snake.dir != 2) snake.dir = 0; // Right (if not going left)

    // Move snake body: shift each segment to position of previous segment
    // Start from tail and work forward to avoid overwriting positions
    for (int i = snake.len - 1; i > 0; i--) {
        snake.x[i] = snake.x[i-1];
        snake.y[i] = snake.y[i-1];
    }

    // Move snake head based on current direction
    if (snake.dir == 0) snake.x[0]++; // Right
    if (snake.dir == 1) snake.y[0]++; // Down
    if (snake.dir == 2) snake.x[0]--; // Left
    if (snake.dir == 3) snake.y[0]--; // Up

    // Check for wall collision (head out of bounds)
    if (snake.x[0] < 0 || snake.x[0] >= GRID_W || snake.y[0] < 0 || snake.y[0] >= GRID_H) {
        snake.gameOver = true;
        return;
    }

    // Check for self-collision (head overlaps with any body segment)
    for (int i = 1; i < snake.len; i++) {
        if (snake.x[0] == snake.x[i] && snake.y[0] == snake.y[i]) {
            snake.gameOver = true;
            return;
        }
    }
}

```

```

}

// Check if snake head reached food
if (snake.x[0] == snake.foodX && snake.y[0] == snake.foodY) {
    // Grow snake if not at maximum length
    if (snake.len < SNAKE_MAX) {
        snake.len++;
    }
    // Increase score
    snake.score += 10;
    // Update high score if current score is higher
    if (snake.score > snakeHighScore) snakeHighScore = snake.score;
    // Spawn new food at random location
    spawnFood();
}
}

/**
 * @brief Render the Snake game to the frame buffer
 *
 * This function draws the complete Snake game screen including:
 * - Game grid border
 * - Snake body segments (head in brighter blue, body in darker blue)
 * - Food item (red circle)
 * - Score and high score display
 * - Game over overlay (if game is over)
 *
 * The function uses mutex protection to safely access the frame buffer,
 * then signals the display task that a new frame is ready.
 *
 * @note Takes bufferMutex before drawing and gives frameSemaphore after drawing
 * @note Head has eyes drawn on it for visual distinction
 */
void drawSnake() {
    // Take mutex to protect frame buffer access
    xSemaphoreTake(bufferMutex, portMAX_DELAY);
    // Clear screen to background color
    Paint_Clear(COLOR_BG);

    // Draw game grid border
    Paint_DrawRectangle(0, 0, GRID_W * GRID_SIZE, GRID_H * GRID_SIZE,
        COLOR_FG, DOT_PIXEL_2X2, DRAW_FILL_EMPTY);

    // Draw each snake segment
    for (int i = 0; i < snake.len; i++) {
        // Convert grid coordinates to pixel coordinates (with 2px padding)
        int px = snake.x[i] * GRID_SIZE + 2;
        int py = snake.y[i] * GRID_SIZE + 2;

        // Head is brighter blue, body is darker blue
        UWORD color = (i == 0) ? 0x001F : 0x0014;

        // Draw snake segment as filled rectangle
        Paint_DrawRectangle(px, py, px + GRID_SIZE - 4, py + GRID_SIZE - 4,
            color, DOT_PIXEL_1X1, DRAW_FILL_FULL);

        // Draw eyes on snake head
        if (i == 0) {
            Paint_DrawRectangle(px + 2, py + 2, px + 3, py + 3,
                COLOR_BG, DOT_PIXEL_1X1, DRAW_FILL_FULL);
            Paint_DrawRectangle(px + 6, py + 2, px + 7, py + 3,
                COLOR_BG, DOT_PIXEL_1X1, DRAW_FILL_FULL);
        }
    }
}

```

```

// Draw food as a red circle
int fx = snake.foodX * GRID_SIZE + 2;
int fy = snake.foodY * GRID_SIZE + 2;
Paint_DrawCircle(fx + GRID_SIZE/2 - 2, fy + GRID_SIZE/2 - 2,
    4, COLOR_FOOD, DOT_PIXEL_1X1, DRAW_FILL_FULL);

// Display score and high score at bottom of screen
char txt[40];
sprintf(txt, "Score:%d Best:%d", snake.score, snake.HighScore);
Paint_DrawString_EN(10, 245, txt, &Font12, COLOR_FG, COLOR_BG);

// Draw game over overlay if game is over
if (snake.gameOver) {
    // Draw semi-transparent overlay background
    Paint_DrawRectangle(40, 90, 200, 150, COLOR_BG, DOT_PIXEL_1X1, DRAW_FILL_FULL);
    // Draw overlay border
    Paint_DrawRectangle(40, 90, 200, 150, COLOR_FG, DOT_PIXEL_2X2, DRAW_FILL_EMPTY);
    // Draw "GAME OVER" text
    Paint_DrawString_EN(70, 105, "GAME OVER", &Font16, COLOR_OBSTACLE, COLOR_BG);
    // Draw final score
    sprintf(txt, "Score: %d", snake.score);
    Paint_DrawString_EN(80, 125, txt, &Font12, COLOR_FG, COLOR_BG);
}

// Release mutex and signal display task that frame is ready
xSemaphoreGive(bufferMutex);
xSemaphoreGive(frameSemaphore);
}

/* ===== MATH QUIZ GAME ===== */

/** @brief Number of questions in the math quiz */
#define MATH QUIZ_QUESTIONS 3

/**
 * @brief Enumeration of math quiz game states
 *
 * Defines the different states the math quiz can be in:
 * - MATH_STATE_PLAYING: Showing question and waiting for answer
 * - MATH_STATE_SHOWING_RESULT: Showing feedback after answering
 * - MATH_STATE_COMPLETE: Quiz finished, showing final score
 */
typedef enum {
    MATH_STATE_PLAYING,
    MATH_STATE_SHOWING_RESULT,
    MATH_STATE_COMPLETE
} MathQuizState;

/**
 * @brief Structure representing a single math question
 *
 * Contains the question data, correct answer, wrong answers, and
 * the shuffled answer array for display.
 */
typedef struct {
    int operand1;          /**< First operand in the math problem */
    int operand2;          /**< Second operand in the math problem */
    char operation;         /**< Operation character ('+' or '-') */
    int correctAnswer;      /**< The correct answer to the problem */
    int wrongAnswer1;       /**< First wrong answer option */
    int wrongAnswer2;       /**< Second wrong answer option */
    int correctIndex;       /**< Which position (0-2) has the correct answer in answers[] */
    int answers[3];         /**< Shuffled answers array for display (correct + 2 wrong) */
} MathQuestion;

```

```

/**
 * @brief Structure representing the complete state of the Math Quiz game
 *
 * Contains current question index, selected answer choice, score,
 * game state, all questions, and feedback timing.
 */
typedef struct {
    int currentQuestion;           /**< Index of current question (0 to MATH_QUIZ_QUESTIONS-1) */
    int selectedChoice;           /**< Currently selected answer choice (0-2) */
    int score;                    /**< Number of correct answers */
    MathQuizState state;         /**< Current game state */
    MathQuestion questions[MATH_QUIZ_QUESTIONS]; /**< Array of all quiz questions */
    bool showingFeedback;        /**< Flag indicating if feedback is being shown */
    TickType_t feedbackStartTime; /**< Tick count when feedback started (for timing) */
} MathQuizGame;

/** @brief Global instance of the Math Quiz game state */
MathQuizGame mathQuiz;

/**
 * @brief Initialize the math quiz questions with fixed problems
 *
 * This function sets up three predefined math questions with double-digit numbers.
 * Each question includes the operands, operation, correct answer, and two wrong
 * answers. The answers are shuffled and stored in the answers[] array with the
 * correctIndex indicating which position contains the correct answer.
 *
 * Questions:
 * 1.  $45 + 32 = 77$  (correct at position 1)
 * 2.  $78 - 24 = 54$  (correct at position 0)
 * 3.  $56 + 19 = 75$  (correct at position 2)
 */
void initMathQuestions() {
    // Question 1:  $45 + 32 = 77$ 
    mathQuiz.questions[0].operand1 = 45;
    mathQuiz.questions[0].operand2 = 32;
    mathQuiz.questions[0].operation = '+';
    mathQuiz.questions[0].correctAnswer = 77;
    mathQuiz.questions[0].wrongAnswer1 = 68;
    mathQuiz.questions[0].wrongAnswer2 = 85;
    // Shuffle answers: correct at position 1
    mathQuiz.questions[0].answers[0] = 68;
    mathQuiz.questions[0].answers[1] = 77; // correct
    mathQuiz.questions[0].answers[2] = 85;
    mathQuiz.questions[0].correctIndex = 1;

    // Question 2:  $78 - 24 = 54$ 
    mathQuiz.questions[1].operand1 = 78;
    mathQuiz.questions[1].operand2 = 24;
    mathQuiz.questions[1].operation = '-';
    mathQuiz.questions[1].correctAnswer = 54;
    mathQuiz.questions[1].wrongAnswer1 = 52;
    mathQuiz.questions[1].wrongAnswer2 = 56;
    // Shuffle answers: correct at position 0
    mathQuiz.questions[1].answers[0] = 54; // correct
    mathQuiz.questions[1].answers[1] = 52;
    mathQuiz.questions[1].answers[2] = 56;
    mathQuiz.questions[1].correctIndex = 0;

    // Question 3:  $56 + 19 = 75$ 
    mathQuiz.questions[2].operand1 = 56;
    mathQuiz.questions[2].operand2 = 19;
    mathQuiz.questions[2].operation = '+';
    mathQuiz.questions[2].correctAnswer = 75;
    mathQuiz.questions[2].wrongAnswer1 = 73;

```



```

    mathQuiz.questions[2].wrongAnswer2 = 77;
    // Shuffle answers: correct at position 2
    mathQuiz.questions[2].answers[0] = 73;
    mathQuiz.questions[2].answers[1] = 77;
    mathQuiz.questions[2].answers[2] = 75; // correct
    mathQuiz.questions[2].correctIndex = 2;
}

/**
 * @brief Initialize the Math Quiz game to starting state
 *
 * This function resets the math quiz to its initial state:
 * - Sets current question to first question (index 0)
 * - Resets selected choice to first option (0)
 * - Resets score to 0
 * - Sets state to playing
 * - Clears feedback flag
 * - Initializes all questions
 */
void initMathQuiz() {
    // Reset to first question
    mathQuiz.currentQuestion = 0;
    // Reset selected choice to first option
    mathQuiz.selectedChoice = 0;
    // Reset score
    mathQuiz.score = 0;
    // Set initial state to playing
    mathQuiz.state = MATH_STATE_PLAYING;
    // Clear feedback flag
    mathQuiz.showingFeedback = false;
    // Initialize all questions
    initMathQuestions();
}

/**
 * @brief Update Math Quiz game state based on input and timing
 *
 * This function processes one game tick of the Math Quiz:
 * - Handles quiz complete state (return to menu on button press)
 * - Manages feedback timing (2 second delay before next question)
 * - Processes navigation input (LEFT/RIGHT to select answer choice)
 * - Handles answer selection (Button A to submit answer)
 * - Updates score based on correct/incorrect answer
 * - Advances to next question or completes quiz
 *
 * @param input Pointer to InputState structure containing current button states
 *
 * @note Feedback is shown for 2 seconds before advancing
 * @note Quiz completes after all questions are answered
 */
void updateMathQuiz(InputState *input) {
    // If quiz is complete, allow return to menu
    if (mathQuiz.state == MATH_STATE_COMPLETE) {
        if (input->btnA) {
            currentGame = GAME_MENU;
        }
        return;
    }

    // If showing feedback, wait for timeout before advancing
    if (mathQuiz.showingFeedback) {
        // Wait 2 seconds before moving to next question
        if (xTaskGetTickCount() - mathQuiz.feedbackStartTime >= pdMS_TO_TICKS(2000)) {
            mathQuiz.showingFeedback = false;
            // Advance to next question

```

```

        mathQuiz.currentQuestion++;
        // Check if all questions completed
        if (mathQuiz.currentQuestion >= MATH_QUIZ_QUESTIONS) {
            mathQuiz.state = MATH_STATE_COMPLETE;
        } else {
            // Reset for next question
            mathQuiz.selectedChoice = 0;
            mathQuiz.state = MATH_STATE_PLAYING;
        }
    }
    return;
}

// Process input during playing state
if (mathQuiz.state == MATH_STATE_PLAYING) {
    // Navigate between answer choices with LEFT/RIGHT buttons
    if (input->btnLeft && mathQuiz.selectedChoice > 0) {
        mathQuiz.selectedChoice--;
    }
    if (input->btnRight && mathQuiz.selectedChoice < 2) {
        mathQuiz.selectedChoice++;
    }

    // Select answer with Button A
    if (input->btnA) {
        MathQuestion *q = &mathQuiz.questions[mathQuiz.currentQuestion];
        int selectedAnswer = q->answers[mathQuiz.selectedChoice];

        // Check if selected answer is correct
        if (mathQuiz.selectedChoice == q->correctIndex) {
            mathQuiz.score++;
        }

        // Start showing feedback
        mathQuiz.showingFeedback = true;
        mathQuiz.feedbackStartTime = xTaskGetTickCount();
        mathQuiz.state = MATH_STATE_SHOWING_RESULT;
    }
}

/**
 * @brief Render the Math Quiz game to the frame buffer
 *
 * This function draws the math quiz screen based on current game state:
 * - Playing state: Shows question, three answer choices (with selection highlight)
 * - Feedback state: Shows question, selected answer, and correct/incorrect feedback
 * - Complete state: Shows final score and return instructions
 *
 * The function uses mutex protection to safely access the frame buffer,
 * then signals the display task that a new frame is ready.
 *
 * @note Takes bufferMutex before drawing and gives frameSemaphore after drawing
 * @note Selected answer choice is highlighted with red background
 */
void drawMathQuiz() {
    // Take mutex to protect frame buffer access
    xSemaphoreTake(bufferMutex, portMAX_DELAY);
    // Clear screen to background color
    Paint_Clear(COLOR_BG);

    if (mathQuiz.state == MATH_STATE_COMPLETE) {
        // Final score screen - quiz is complete
        Paint_DrawString_EN(50, 50, "Quiz Complete!", &Font20, COLOR_FG, COLOR_BG);
    }
}

```

```

char txt[40];
sprintf(txt, "Score: %d/%d", mathQuiz.score, MATH_QUIZ_QUESTIONS);
Paint_DrawString_EN(70, 100, txt, &Font20, COLOR_FG, COLOR_BG);

// Instructions to return to menu
Paint_DrawString_EN(40, 180, "Press A to return", &Font12, COLOR_FG, COLOR_BG);
Paint_DrawString_EN(40, 200, "B - Back to Menu", &Font12, COLOR_FG, COLOR_BG);
} else if (mathQuiz.showingFeedback) {
    // Show feedback after answering
    MathQuestion *q = &mathQuiz.questions[mathQuiz.currentQuestion];
    bool isCorrect = (mathQuiz.selectedChoice == q->correctIndex);

    char txt[60];
    // Show question number
    sprintf(txt, "Question %d/%d", mathQuiz.currentQuestion + 1, MATH_QUIZ_QUESTIONS);
    Paint_DrawString_EN(50, 30, txt, &Font16, COLOR_FG, COLOR_BG);

    // Show the math problem
    sprintf(txt, "%d %c %d = ?", q->operand1, q->operation, q->operand2);
    Paint_DrawString_EN(60, 80, txt, &Font20, COLOR_FG, COLOR_BG);

    // Show feedback based on correctness
    if (isCorrect) {
        Paint_DrawString_EN(80, 130, "Correct!", &Font20, COLOR_OBSTACLE, COLOR_BG);
    } else {
        Paint_DrawString_EN(70, 130, "Incorrect!", &Font20, COLOR_OBSTACLE, COLOR_BG);
        sprintf(txt, "Answer: %d", q->correctAnswer);
        Paint_DrawString_EN(70, 160, txt, &Font16, COLOR_FG, COLOR_BG);
    }
} else {
    // Show question and answer choices (playing state)
    MathQuestion *q = &mathQuiz.questions[mathQuiz.currentQuestion];

    char txt[60];
    // Show question number
    sprintf(txt, "Question %d/%d", mathQuiz.currentQuestion + 1, MATH_QUIZ_QUESTIONS);
    Paint_DrawString_EN(50, 20, txt, &Font16, COLOR_FG, COLOR_BG);

    // Show the math problem
    sprintf(txt, "%d %c %d = ?", q->operand1, q->operation, q->operand2);
    Paint_DrawString_EN(60, 60, txt, &Font20, COLOR_FG, COLOR_BG);

    // Show three answer choices
    int yStart = 120;
    int ySpacing = 35;

    for (int i = 0; i < 3; i++) {
        int y = yStart + i * ySpacing;
        UWORD bgColor = COLOR_BG;
        UWORD textColor = COLOR_FG;

        // Highlight selected choice with red background
        if (i == mathQuiz.selectedChoice) {
            bgColor = COLOR_OBSTACLE;
            textColor = COLOR_BG;
            // Draw background rectangle for selected choice
            Paint_DrawRectangle(70, y - 2, 170, y + 18, bgColor, DOT_PIXEL_1X1, DRAW_FILL_FULL);
        }

        // Draw answer choice label and value
        sprintf(txt, "%c %d", 'A' + i, q->answers[i]);
        Paint_DrawString_EN(80, y, txt, &Font16, textColor, bgColor);
    }

    // Display control instructions

```

```

    Paint_DrawString_EN(30, 230, "LEFT/RIGHT - Select A - Answer", &Font12, COLOR_FG, COLOR_BG);
}

// Release mutex and signal display task that frame is ready
xSemaphoreGive(bufferMutex);
xSemaphoreGive(frameSemaphore);
}

/* ===== MENU SYSTEM ===== */

/** @brief Currently selected menu item index (0 = Math Quiz, 1 = Snake, 2 = Power Off) */
int menuSelection = 0;

/**
 * @brief Update menu state based on input
 *
 * This function processes menu navigation and selection:
 * - UP/DOWN buttons navigate between menu items (0-2)
 * - Button A or CTRL selects the current menu item
 * - Menu item 0: Start Math Quiz game
 * - Menu item 1: Start Snake game
 * - Menu item 2: Power off (clear screen, disable backlight, enter infinite loop)
 *
 * @param input Pointer to InputState structure containing current button states
 *
 * @note Power off option clears the screen, disables backlight, and enters infinite loop
 * @note Menu selection range is 0-2 (3 options)
 */
void updateMenu(InputState *input) {
    // Navigate menu with UP/DOWN buttons (range 0-2)
    if (input->btnUp && menuSelection > 0) menuSelection--;
    if (input->btnDown && menuSelection < 2) menuSelection++;

    // Handle menu selection with Button A or CTRL
    if (input->btnA || input->btnCtrl) {
        if (menuSelection == 0) {
            // Start Math Quiz game
            currentGame = GAME_MATH_QUIZ;
            initMathQuiz();
        } else if (menuSelection == 1) {
            // Start Snake game
            currentGame = GAME_SNAKE;
            initSnake();
        } else if (menuSelection == 2) {
            // Power off: clear screen, disable backlight, stop execution
            xSemaphoreTake(bufferMutex, portMAX_DELAY);
            Paint_Clear(BLACK); // Clear screen to black
            LCD_1IN3_Display(frameBuffer); // Display black screen
            DEV_SET_PWM(0); // Disable backlight
            xSemaphoreGive(bufferMutex);

            // Enter infinite loop to stop all logic
            while(1) {
                vTaskDelay(pdMS_TO_TICKS(1000));
            }
        }
    }
}

/**
 * @brief Render the menu screen to the frame buffer
 *
 * This function draws the main menu screen including:
 * - Menu title "GAME MENU"
 * - Three menu options (Math Quiz, Snake, Power Off)
 */

```

```

* - Highlighted selection (selected item shown in red)
* - Control instructions at bottom
*
* The function uses mutex protection to safely access the frame buffer,
* then signals the display task that a new frame is ready.
*
* @note Takes bufferMutex before drawing and gives frameSemaphore after drawing
* @note Selected menu item is highlighted with red (COLOR_OBSACLE) text
*/
void drawMenu() {
    // Take mutex to protect frame buffer access
    xSemaphoreTake(bufferMutex, portMAX_DELAY);
    // Clear screen to background color
    Paint_Clear(COLOR_BG);

    // Draw menu title
    Paint_DrawString_EN(40, 30, "GAME MENU", &Font24, COLOR_FG, COLOR_BG);

    // Draw menu options with selection highlighting
    // Selected item is shown in red (COLOR_OBSACLE), others in black (COLOR_FG)
    Paint_DrawString_EN(30, 80, "1. Math Quiz", &Font20,
        menuSelection == 0 ? COLOR_OBSACLE : COLOR_FG, COLOR_BG);
    Paint_DrawString_EN(30, 110, "2. Snake", &Font20,
        menuSelection == 1 ? COLOR_OBSACLE : COLOR_FG, COLOR_BG);
    Paint_DrawString_EN(30, 140, "3. Power Off", &Font20,
        menuSelection == 2 ? COLOR_OBSACLE : COLOR_FG, COLOR_BG);

    // Draw control instructions at bottom of screen
    Paint_DrawString_EN(30, 180, "UP/DOWN - Select", &Font12, COLOR_FG, COLOR_BG);
    Paint_DrawString_EN(30, 200, "A - Start Game", &Font12, COLOR_FG, COLOR_BG);
    Paint_DrawString_EN(30, 220, "B - Back to Menu", &Font12, COLOR_FG, COLOR_BG);

    // Release mutex and signal display task that frame is ready
    xSemaphoreGive(bufferMutex);
    xSemaphoreGive(frameSemaphore);
}

/* ===== FreeRTOS Tasks ===== */

/**
* @brief FreeRTOS task that reads button inputs with debouncing
*
* This task continuously polls all button GPIO pins and sends input state
* changes to the input queue. It implements debouncing by requiring a minimum
* time interval (DEBOUNCE_TIME) between state changes before sending updates.
*
* The task:
* - Reads all button states from GPIO pins
* - Compares current state with previous state
* - Only sends to queue if state changed and debounce time elapsed
* - Updates last known state and debounce timer
*
* @param pvParameters Task parameters (unused)
*
* @note Buttons are active-low (inverted logic: !DEV_Digital_Read)
* @note Debounce time is DEBOUNCE_TIME milliseconds (50ms default)
* @note Task runs at 10ms interval for responsive input
* @note Queue sends are non-blocking (timeout 0)
*/
void inputTask(void *pvParameters) {
    InputState input = {0}, lastInput = {0};
    TickType_t lastDebounce = 0;

    // Infinite loop - task runs forever
    while (1) {

```

```

TickType_t now = xTaskGetTickCount();

// Check if debounce time has elapsed since last state change
if (now - lastDebounce >= pdMS_TO_TICKS(DEBOUNCE_TIME)) {
    // Read all button states (buttons are active-low, so invert)
    input.btnA = !DEV_Digital_Read(BTN_A);
    input.btnB = !DEV_Digital_Read(BTN_B);
    input.btnUp = !DEV_Digital_Read(BTN_UP);
    input.btnDown = !DEV_Digital_Read(BTN_DOWN);
    input.btnLeft = !DEV_Digital_Read(BTN_LEFT);
    input.btnRight = !DEV_Digital_Read(BTN_RIGHT);
    input.btnCtrl = !DEV_Digital_Read(BTN_CTRL);

    // Only send to queue if input state changed
    if (memcmp(&input, &lastInput, sizeof(InputState)) != 0) {
        // Send new input state to queue (non-blocking)
        xQueueSend(inputQueue, &input, 0);
        // Update last known state
        lastInput = input;
        // Reset debounce timer
        lastDebounce = now;
    }
}

// Delay before next button check (10ms = 100 checks per second)
vTaskDelay(pdMS_TO_TICKS(10));
}
}

/**
 * @brief FreeRTOS task that handles game logic and rendering
 *
 * This task is the main game loop that:
 * - Receives input events from the input queue
 * - Handles global input (Button B returns to menu from any game)
 * - Updates current game state based on game type
 * - Renders current game screen
 * - Maintains consistent frame timing using vTaskDelayUntil
 *
 * The task switches between different game modes:
 * - GAME_MENU: Menu navigation and selection
 * - GAME_MATH_QUIZ: Math quiz game logic and rendering
 * - GAME_SNAKE: Snake game logic and rendering
 *
 * @param pvParameters Task parameters (unused)
 *
 * @note Uses vTaskDelayUntil for consistent frame timing
 * @note Frame rate is 150ms per frame (~6.7 FPS) for menu and games
 * @note Button B always returns to menu from any game
 * @note Queue receives are non-blocking (timeout 0)
 */
void gameLogicTask(void *pvParameters) {
    InputState input = {0};
    TickType_t lastWakeTime = xTaskGetTickCount();

    // Infinite loop - task runs forever
    while (1) {
        // Check for input events (non-blocking)
        if (xQueueReceive(inputQueue, &input, 0) == pdTRUE) {
            // Button B always returns to menu from any game
            if (input.btnB && currentGame != GAME_MENU) {
                currentGame = GAME_MENU;
            }
        }
    }
}

```

```

// Update and render based on current game type
if (currentGame == GAME_MENU) {
    updateMenu(&input);
    drawMenu();
    // Maintain 150ms frame time (~6.7 FPS)
    vTaskDelayUntil(&lastWakeTime, pdMS_TO_TICKS(150));
} else if (currentGame == GAME_MATH_QUIZ) {
    updateMathQuiz(&input);
    drawMathQuiz();
    // Maintain 150ms frame time (~6.7 FPS)
    vTaskDelayUntil(&lastWakeTime, pdMS_TO_TICKS(150));
} else if (currentGame == GAME_SNAKE) {
    updateSnake(&input);
    drawSnake();
    // Maintain 150ms frame time (~6.7 FPS)
    vTaskDelayUntil(&lastWakeTime, pdMS_TO_TICKS(150));
}
}
}

/**
 * @brief FreeRTOS task that handles LCD display updates
 *
 * This task waits for frame ready signals from game logic tasks and
 * transfers the frame buffer to the LCD display. It uses semaphores
 * to synchronize with game rendering:
 * - Waits for frameSemaphore (signaled when new frame is ready)
 * - Takes bufferMutex to safely access frame buffer
 * - Transfers frame buffer to LCD display hardware
 * - Releases mutex
 *
 * This decouples display timing from game logic, allowing games to
 * render at their own pace while display updates happen asynchronously.
 *
 * @param pvParameters Task parameters (unused)
 *
 * @note Blocks on frameSemaphore until game logic signals new frame
 * @note Uses mutex to protect frame buffer during LCD transfer
 * @note Lower priority than game logic task to avoid blocking rendering
 */
void displayTask(void *pvParameters) {
    // Infinite loop - task runs forever
    while (1) {
        // Wait for frame ready signal from game logic task
        if (xSemaphoreTake(frameSemaphore, portMAX_DELAY) == pdTRUE) {
            // Take mutex to protect frame buffer during LCD transfer
            xSemaphoreTake(bufferMutex, portMAX_DELAY);
            // Transfer frame buffer to LCD display hardware
            LCD_1IN3_Display(frameBuffer);
            // Release mutex
            xSemaphoreGive(bufferMutex);
        }
    }
}

/* ===== Main Function ===== */

/**
 * @brief Main function that initializes hardware and starts FreeRTOS scheduler
 *
 * This function performs the following initialization steps:
 * 1. Initializes stdio for serial communication
 * 2. Initializes WaveShare LCD display module and graphics buffer
 * 3. Configures all button GPIO pins
 * 4. Creates FreeRTOS queues and semaphores for inter-task communication

```

- * 5. Creates and starts all FreeRTOS tasks
- * 6. Starts the FreeRTOS scheduler
- *
- * Tasks created:
 - * - Input task (priority 3): Reads button inputs with debouncing
 - * - Game Logic task (priority 2): Handles game state and rendering
 - * - Display task (priority 1): Updates LCD display hardware
- *
- * @return int Exit status (never returns due to infinite scheduler loop)
- *
- * @note Returns -1 on initialization failure (display or memory allocation)
- * @note Never returns under normal operation (scheduler runs forever)
- */

```
int main() {
    // === 1) Initialize Standard I/O ===
    // Initialize standard I/O (UART for debugging and serial communication)
    stdio_init_all();
    // Small delay to allow UART to stabilize
    DEV_Delay_ms(100);

    // === 2) Initialize Waveshare LCD Display Module ===
    // Initialize the Waveshare device module (SPI, GPIO, etc.)
    if (DEV_Module_Init() != 0) return -1;

    // Set LCD backlight PWM to 70% brightness
    DEV_SET_PWM(70);
    // Initialize LCD display hardware in horizontal orientation
    LCD_1IN3_Init(HORIZONTAL);
    // Clear LCD to white
    LCD_1IN3_Clear(WHITE);

    // === 3) Allocate Display Frame Buffer ===
    // Calculate frame buffer size: width * height * 2 bytes per pixel (RGB565)
    UDOUBLE imageSize = LCD_1IN3_HEIGHT * LCD_1IN3_WIDTH * 2;
    // Allocate memory for frame buffer
    frameBuffer = (UWORD *)malloc(imageSize);
    if (!frameBuffer) return -1;

    // Initialize graphics library with frame buffer
    Paint_NewImage((UBYTE *)frameBuffer, LCD_1IN3.WIDTH, LCD_1IN3.HEIGHT, 0, WHITE);
    // Set font scale to 65
    Paint_SetScale(65);
    // Set rotation to 0 degrees (normal orientation)
    Paint_SetRotate(ROTATE_0);

    // === 4) Configure Button GPIO Pins ===
    // Configure all button pins for input
    DEV_KEY_Config(BTN_A);
    DEV_KEY_Config(BTN_B);
    DEV_KEY_Config(BTN_UP);
    DEV_KEY_Config(BTN_DOWN);
    DEV_KEY_Config(BTN_LEFT);
    DEV_KEY_Config(BTN_RIGHT);
    DEV_KEY_Config(BTN_CTRL);

    // === 5) Create FreeRTOS Queues and Semaphores ===
    // Create queue for input state messages (5 messages capacity)
    inputQueue = xQueueCreate(5, sizeof(InputState));
    // Create binary semaphore for frame ready signaling
    frameSemaphore = xSemaphoreCreateBinary();
    // Create mutex for frame buffer protection
    bufferMutex = xSemaphoreCreateMutex();

    // === 6) Create FreeRTOS Tasks ===
    // Create input task: reads buttons, sends to queue (priority 3, stack 256 bytes)
```



```

xTaskCreate(inputTask, "Input", 256, NULL, 3, NULL);
// Create game logic task: updates games, renders frames (priority 2, stack 1024 bytes)
xTaskCreate(gameLogicTask, "Logic", 1024, NULL, 2, NULL);
// Create display task: updates LCD hardware (priority 1, stack 256 bytes)
xTaskCreate(displayTask, "Display", 256, NULL, 1, NULL);

// === 7) Start FreeRTOS Scheduler ===
// Start the FreeRTOS real-time scheduler (never returns)
vTaskStartScheduler();

// This code should never be reached
return 0;
}

```

Висновок: У ході лабораторної роботи було створено багатозадачну міні-консоль на FreeRTOS, де кожна частина системи жила у своєму власному ритмі: опитування кнопок, ігрова логіка та рендер кадрів. Вдалося реалізувати черги, семафори і м'ютекси, перетворивши хаос апаратних подій на кероване і пружне ігрове середовище. До консолі було додано меню, паузу та кілька повноцінних ігор, що реагують на користувача в реальному часі. Робота показала, як RTOS дозволяє створювати плавні, швидкі і чітко синхронізовані інтерактивні застосунки на мікроконтролерах.